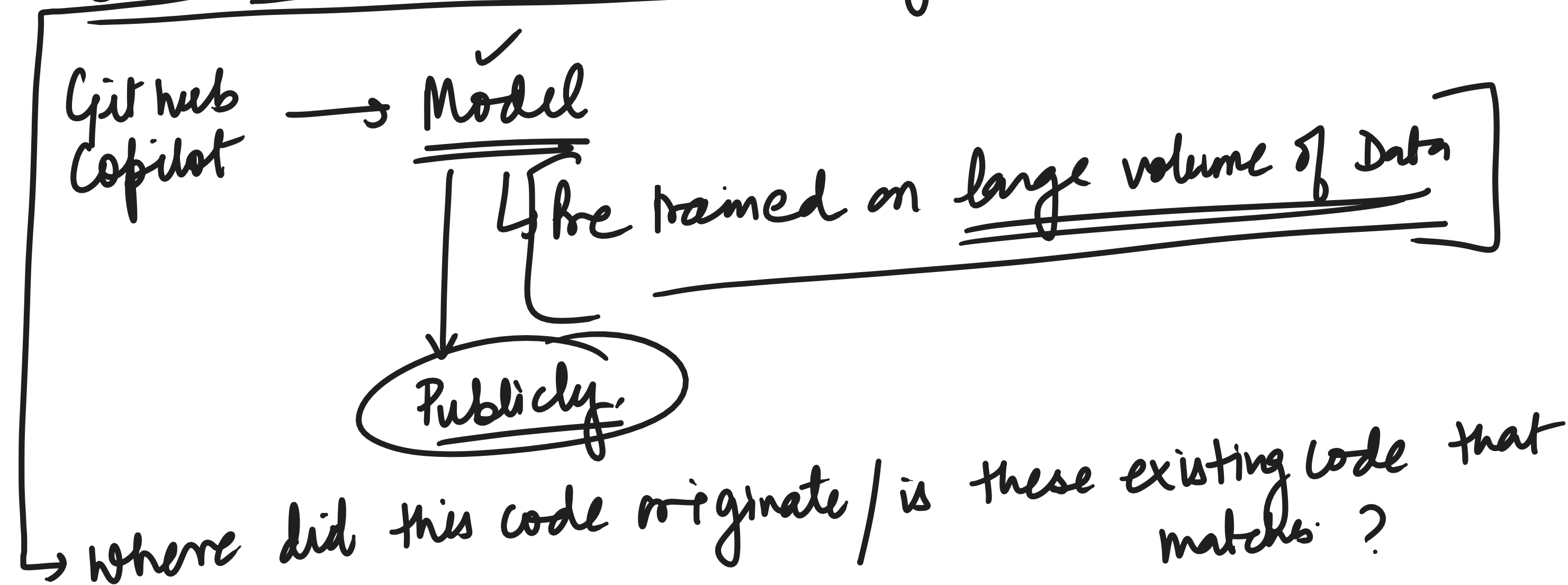


Day 3: Agenda

- ✓ Governance, Responsible usage, BPP, Stay current with Copilot updates
- ✓ Capstone - Employee Leave Management Rest API
- Doc = Challenges -
- Capstone with Solution

80%

'Code Provenance' & Licensing Awareness



Responsible AI usage:

- ① What context are we giving Copilot?
- ② What output is Copilot producing?
- ③ What happens before that output reaches production?

Principle 1 - Protect Sensitive Info

- 2 - Verify the output
- 3 - Apply Normal Engineering Controls
- 4 - Maintain Human Accountability
- 5 - Give Copilot Appropriate Context

What Copilot CAN & CANNOT do?

- CAN
- ✓ Code generation
 - ✓ Boilerplate generation
 - ✓ Explaining unfamiliar code
 - ✓ Refactoring
 - ✓ Writing Tests
 - ✓ generating documentation
 - ✓ debugging assistance
 - ✓ suggesting API usage
 - ✓ understanding Repo context
 - ✓ Review of code
 - ✓ Helping you explore alternative implementations

- CANNOT
- Generated code:
- ✓ Correct
 - ✓ secure
 - ✓ optimized
 - ✓ compliant
 - ✓ properly licensed
 - ✓ Architecturally appropriate
 - ✓ compatible with every internal standards
 - ✓ business Requirements

Still Require Human

Best Practices

Context → Generate → Inspect → Validate → Integrate

- ① Context
 - ✓ what you want
 - ✓ Important Rules
 - ✓ Technologies
 - ✓ Expected Behavior
 - ✓ Patterns already used by the project

② Generate

- ③ Inspect
 - ✓ Do I understand it?
 - ✓ Does it solve the requested problem?
 - ✓ Did it introduce an unnecessary dependency?
 - ✓ Did it expose credentials?
 - ✓ Did it change behavior I didn't request?
 - ✓ Is there a public-code reference?
 - ✓ Is it following our code conventions?

- ④ Validate → Run
 - Compilation
 - unit tests
 - integration tests
 - security checks
 - dependency scanning

- ⑤ Integrate
 - ✓ source control
 - ✓ PR - pull request
 - ✓ Peer Review
 - ✓ CI/CD
 - ✓ approval gates
 - ✓ deployment controls